E-Commerce Order Management System

Microservices Architecture Project Specification

Version: 1.0
Date: 2026-07-02
Author: Z.ai

1. Project Overview

Objective:

Build a backend system for an e-commerce platform that handles product catalog management, order processing, and inventory tracking using a Microservices architecture.

Goal:

Demonstrate proficiency in Core Java, Spring Boot, Microservices, DevOps, Security, and Software Engineering Principles (TDD, DDD, SOLID).

2. Architecture & Tech Stack Mapping

This section maps your required skills directly to the project components.

Component	Technology/Approach
Architecture	Microservices, Domain-Driven Design (DDD)
Language & Framework	Core Java / Java EE, Spring Boot, Spring Cloud
Database	Oracle Database (Oracle XE via Docker)
API Design	RESTful APIs
Principles	OOP, SOLID, KISS, DRY, TDD/BDD
Testing	JUnit, Mockito
Security	Spring Security, JWT (Backend Security)
Operations	SLF4J/Logback (Logging), Spring Actuator (Monitoring)
DevOps & Tools	Git, Docker, CI/CD (GitHub Actions), Jira, Scrum

3. Microservices Breakdown

We will build 3 independent microservices representing bounded contexts in Domain-Driven Design (DDD):

Product Service: Manages products, prices, and categories.

Inventory Service: Manages stock levels for products.

Order Service: Handles customer orders, communicates with Inventory to check stock.

4. Step-by-Step Execution Plan (Sprints)

Sprint 1: Planning, Setup & Architecture

Skills: Scrum, Jira, Git, DDD, Architecture

Task 1: Create a Jira project. Write Epics (e.g., 'Product Service API', 'CI/CD Pipeline') and break them down into User Stories and Tasks. Move tasks across the board (To Do → In Progress → Done).

Task 2: Initialize a Git repository (GitHub/GitLab). Create branches: main, develop, feature/product-service.

Task 3: Define Bounded Contexts (DDD). Identify aggregates (Product, Order, Inventory) and their relationships.

Task 4: Design the REST APIs (e.g., POST /api/v1/products, GET /api/v1/orders/{id}).

Sprint 2: Core Development & Database

Skills: Core Java, OOP, Spring, REST, Oracle DB, SOLID, KISS, DRY

Task 1: Setup Oracle Database (Use Docker image gvenzl/oracle-xe to run Oracle locally).

Task 2: Create the Product Service Spring Boot application.

Task 3: Implement Entities using OOP principles (Encapsulation, Inheritance if needed). Use Spring Data JPA to connect to Oracle DB.

Task 4: Create DTOs (Data Transfer Objects) and Mappers (MapStruct) to separate DB entities from API responses (Applying SOLID - Single Responsibility Principle).

Task 5: Implement Service and Controller layers. Apply KISS and DRY (avoid duplicate validation logic, keep methods small and focused).

Sprint 3: Quality & Testing

Skills: JUnit, TDD, BDD, Exception Handling

Task 1: Write tests first for the Order Service using TDD:

Task 2: • Red: Write a JUnit test for createOrder() that fails because the method doesn't exist.

Task 3: • Green: Write the minimum code in the Service layer to make the test pass.

Task 4: • Refactor: Clean up the code.

Task 5: Use BDD naming conventions for tests: givenProductExists_whenCreateOrder_thenReturnOrderDto().

Task 6: Implement Global Exception Handling using @ControllerAdvice. Create custom exceptions (e.g., ResourceNotFoundException, OutOfStockException) and return standardized JSON error responses.

Sprint 4: Security & Observability

Skills: Backend Security, Logging, Monitoring

Task 1: Implement Security using Spring Security and JWT. Secure the POST/PUT/DELETE endpoints so only authenticated Admins can access them, while GET endpoints are public.

Task 2: Add Logging using SLF4J. Log important events (e.g., log.info('Order created successfully with id: {}', order.getId())) and errors (log.error('Failed to process order due to insufficient stock')).

Task 3: Add Monitoring using Spring Boot Actuator. Expose /actuator/health and /actuator/metrics to monitor application status.

Sprint 5: Inter-Service Communication

Skills: Microservices

Task 1: Make Order Service talk to Inventory Service to check stock before placing an order. Use OpenFeign or RestTemplate.

Task 2: Handle communication failures gracefully (e.g., if Inventory Service is down, the Order Service should not crash; implement a fallback/Circuit Breaker concept).

Sprint 6: DevOps & Deployment

Skills: Docker, CI/CD

Task 1: Dockerize each microservice. Write a Dockerfile for the Product, Order, and Inventory services. Write a docker-compose.yml to spin up the 3 services plus the Oracle Database together.

Task 2: Create a CI/CD Pipeline using GitHub Actions (or GitLab CI):

• Step 1: Trigger on push to main or PR.

• Step 2: Setup Java 17.

• Step 3: Run mvn clean test (automated testing!).

• Step 4: Build the .jar file.

• Step 5: Build the Docker image and push to Docker Hub.